ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY & SCIENCES
(AUTONOMOUS)
(Permanent Affiliation by Andhra University & Approved by AICTE
Accredited by NBA & NAAC with A+Grade)
Sangivalasa-531162, Bheemunipatnam Mandal,
Visakhapatnam District

A PROJECT WORK REPORT ON
DIGITAL DIARY/PERSONAL ORGANIZER

This project is submitted by

GORLE HARSHA VARDHAN	A25126510015
POOJA SINGH	A25126510040
PUPPALA SRI VARSHINI	A25126510041
SIRASAPALLI BHAVAN	A25126510051

Under the Guidance of
Dr B. MAHESH

Department of Computer Science and Engineering in partial fulfilment of the requirements for the award of the offered by
ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY AND SCIENCES

Bachelor of Technology
in
COMPUTER SCIENCE AND ENGINEERING
2025-2029

ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY & SCIENCES

(AUTONOMOUS)

(Permanent Affiliation by Andhra University & Approved by AICTE
Accredited by NBA & NAAC with A+ Grade)
Sangivalasa-531162, Bheemunipatnam Mandal, Visakhapatnam District



CERTIFICATE

This is to certify that this project work entitled - **"DIGITAL DIARY/PERSONAL ORGANIZER"** is the bonafide record work done by **Mr. Gorle Harsha Vardhan, Ms. Pooja Singh, Ms. Puppala Sri Varshini, Mr. Sirasapalli Bhavan**bearing Roll no: **A25126510015, A25126510040, A25126510041, A25126510051** of the First year, along with batch mates submitted on the partial fulfilment of the requirement for the award of B.Tech in Computer Science and Engineering to the ANIL NEEKURONDA INSTITUTE OF TECHNOLOGY AND SCIECNCES. The results embodied in this project report have not been submitted to any other board/University or Institute for the award of diploma.

.

HEAD OF THE DEPARTMENT LECTURER IN CHARGE

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to all those who have contributed to the successful completion of this project. First and foremost, I extend my heartfelt thanks to my project guide, Dr. B. MAHESH, for their valuable guidance, continuous support, and encouragement throughout the development of this project. Their insightful feedback and expert advice were instrumental in shaping this work.

I am also grateful to the faculty members of Computer Science and Engineering, ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY AND SCIENCES, for providing the necessary resources and a conducive environment for research and learning.

A special thanks to my friends and classmates for their cooperation, constructive suggestions, and moral support during every stage of this project. Finally, I would like to thank my family for their unwavering love, patience, and constant motivation that kept me focused and determined. This project, titled “DIGITAL DIARY/PERSONAL ORGANIZER”, has been a great learning experience, helping me enhance my technical skills and practical understanding of the subject.

ABSTRACT

The Digital Diary Entry System is a console-based application developed using the C++ programming language that allows users to store and manage personal diary entries securely in a digital format. The system is designed to replace traditional handwritten diaries by providing an organized, efficient, and easily accessible solution for recording daily thoughts and events.
The application includes a password-based authentication mechanism to prevent unauthorized access. Users can perform essential operations such as adding new diary entries, viewing existing entries, editing previously saved content, and deleting unwanted records. File handling techniques are used to store diary data permanently, ensuring that information is retained even after the program is closed.

This project demonstrates the practical implementation of object-oriented programming concepts, file handling, and menu-driven program design in C++. The Digital Diary Entry System is simple to use, reliable, and suitable for academic purposes. It also provides a strong foundation for future enhancements such as advanced security features, database integration, and graphical user interface development.

PREFACE

Table of contents

Sl.No.	Chapter Title	Sub-Contents	Page Number
1	Introduction	• Overview of the Project • Purpose and Scope • Existing System and Proposed System	1-2
2	Problem Definition	• Problem Statement • Goals of the System	2-3
3	System Requirements	• Hardware requirements • Software requirements	3
4	Features	• Features of the system	3-4
5	Advantages	• Advantages of the System	4
6	System Design	• System Architecture • Flowcharts • Algorithms Used	4-7

7	Implementation Details	• Program Structure • Language and Environment Used • Key Functional Modules • Code Snippets and Screenshots • File Handling Explanation	8-16
8	Results and Discussion	• Output Screens • Test Case Table (Input, Expected Output, Actual Output)	16-21
9	Conclusion	• Summary of Achievements • Limitations of the System • Future Enhancements	21-22
10	References	• Books Referred • Research Papers • Websites and Online Resources	22
11	Appendix	• Source Code Listings • Source Code	22-26

DIGITAL DIARY/PERSONAL ORGANIZER(C++)

1.INTRODUCTION

Overview of the Project:

This project is about making a digital diary system using C++. It's basically a console program where people can write down their thoughts instead of using paper. You can add entries, look at them, change stuff, or delete if you want. Each entry has a date, and everything gets saved in a file named diary.txt so it does not disappear when you close the program. There is also a login with a password to keep others out, which I think makes it more secure.

The whole point here is to give users an easy way to store daily stuff digitally, without all the hassle of physical notebooks. Old diaries on paper have problems, you know. They wear out over time from just handling them. Once you write something, editing is not really an option, and you have to carry it around physically. It seems like those issues make the traditional way outdated for most people.

This project demonstrates the practical use of Object-Oriented Programming concepts and file management in C++, showing how real-world problems can be solved using programming fundamentals.

Purpose of the Project:

- The main purpose of the Digital Diary System is to provide a secure and reliable digital platform for maintaining personal records.
- It aims to protect user privacy by restricting access through authentication.
- The system reduces dependence on physical diaries, which are prone to loss and damage.
- It enables users to manage their diary entries easily without rewriting content.
- The project is designed to simplify record keeping and improve data accessibility.
- It helps users save time by allowing quick retrieval of stored entries.
- The system also serves as a learning tool to understand real-world application development using C++.

Scope of the Project:

- The project supports digital storage of diary entries on a computer system.
- It allows users to create new records with date-wise organization.
- Users can edit previously stored diary entries whenever required.
- The system provides an option to delete outdated or unnecessary records.
- Password-based security ensures only authorized access.
- All data is permanently stored using file handling techniques.
- The application uses a menu-driven interface that is simple and easy to navigate.
- The scope is limited to a single-user environment.
- The project can be extended in the future to include multiple users, databases, encryption, or a graphical interface.

- It provides foundational knowledge useful for advanced software development.

Existing System:

The traditional diary system has several limitations, including:

- Physical damage due to wear and tear over time
- No protection or encryption for personal information
- Difficulty in searching for specific entries
- No option to modify content once written
- Requires manual effort and physical presence

Proposed System:

The Digital Diary System is designed to overcome the limitations of conventional diaries by providing:

- Secure access through password authentication
- Easy creation, modification, and deletion of diary entries
- Organized and systematic storage of data in files
- Faster access and better management of personal records.

2.PROBLEM DEFINITION

Problem Statement:

Many individuals still rely on handwritten diaries to record their daily activities, personal thoughts, and important events.

While this method is familiar, it presents several challenges such as lack of privacy, difficulty in organizing information, and risk of data loss due to physical damage or misplacement. Manual diaries do not provide any secure access control, allowing unauthorized people to read personal content.

Searching for specific past entries is time-consuming, and modifying existing records often requires rewriting entire pages.

In the current digital age, where computers and electronic devices are widely used, these limitations make traditional diary systems inefficient and unreliable. There is a need for a digital solution that allows users to safely store personal entries, quickly retrieve previous records, and easily update or delete information when required.

Therefore, the problem is to design and develop a secure and efficient Digital Diary Entry System that provides authentication, organized data storage, and permanent record management using file handling techniques.

Goals of the System:

- To develop a digital diary application using C++ • To provide secure access through a login system
- To support basic CRUD operations:

- Create diary entries
- Read saved entries
- Update existing entries
- Delete unwanted entries
- To store data permanently using file handling • To ensure a simple and user-friendly interface

3.SYSTEM REQUIREMENTS

Hardware Requirements:

1. Computer or Laptop
2. Minimum 4 GB RAM
3. Keyboard and Mouse

Software Requirements:

1. Operating System: Windows / Linux
2. Compiler: Dev C++ / Code::Blocks / VS Code
3. Programming Language: C++

4.FEATURES OF THE SYSTEM

The Digital Diary Entry System is designed with several features that enhance efficiency, security, and ease of use. One of the key features of the system is the ability to create diary entries by allowing users to record their daily activities, personal thoughts, and important events along with the corresponding date. This date-wise organization helps maintain a clear and structured record of information. The system also provides a viewing feature that enables users to access all previously stored diary entries at any time, making it convenient to recall past memories or reference earlier records quickly.

Another important feature of the system is its editing functionality, which allows users to modify existing diary entries whenever updates are required. This eliminates the need to rewrite entire records, offering greater flexibility compared to traditional handwritten diaries. In addition, the system includes a delete option that allows users to remove unwanted or outdated entries, helping maintain relevant and organized data.

To ensure privacy and data protection, the system incorporates password-based authentication. Only authorized users can access the diary, which prevents unauthorized viewing or misuse of personal information. File handling techniques are used to store diary entries permanently in files, ensuring that data is not lost when the program is closed and can be retrieved whenever needed. The application also features a simple, menu-driven interface that guides users through various operations such as adding, viewing, editing, and deleting entries. This interface makes the system

easy to use even for individuals with basic computer knowledge. Overall, the Digital Diary Entry System is a reliable and user-friendly solution for digital record keeping and can be extended in the future to include advanced features such as encryption, multiple user support, or a graphical user interface.

5.ADVANTAGES OF THE SYSTEM

The Digital Diary Entry System provides numerous advantages when compared to conventional handwritten diary methods. One of the major benefits of the system is enhanced data security. Through password-based authentication, the system ensures that only authorized users can access diary entries, thereby protecting personal information from unauthorized viewing and misuse. Another significant advantage is time efficiency. The system allows users to quickly add, view, edit, or delete diary entries without manual effort. Unlike traditional diaries, where searching for a specific entry can be time-consuming, the digital system enables faster access to stored information, improving overall productivity.

The system also eliminates the need for paper-based record keeping, which reduces physical storage requirements and minimizes clutter. Since diary entries are stored digitally, the risk of data loss due to physical damage such as tearing, fire, or water exposure is greatly reduced. Additionally, file handling ensures permanent data storage, allowing users to retrieve their records even after the application is closed.

From a practical perspective, the Digital Diary Entry System is cost-effective and user-friendly. It does not require expensive hardware or specialized software and can run efficiently on basic computer systems. The menu-driven interface makes it easy to use, even for beginners with limited technical knowledge.

Furthermore, the system serves as an effective learning tool for students. It helps in understanding real-world implementation of programming concepts such as file handling, object-oriented programming, data management, and basic security mechanisms. Overall, the Digital Diary Entry System offers a reliable, efficient, and secure solution for personal record management while also supporting educational objectives.

6. SYSTEM DESIGN

System Architecture:

User Interface Layer

Provides a console-based interface for user interaction, including login, menu selection, and data input.

- Application Logic Layer

Contains the core logic implemented using the DiaryEntry class and its member functions for adding, viewing, editing, and deleting diary entries.

- Authentication Layer

Handles password verification to ensure secure access before performing diary operations.

• Data Storage Layer
Manages persistent storage of diary entries using a text file (diary.txt) through file handling operations.

• File Management Layer
Supports reading, writing, updating, and deleting records using temporary files to ensure data consistency.

ALGORITHM:

Algorithm 1: Login Authentication

1. Start the program.
2. Display the login screen.
3. Prompt the user to enter the password
4. Compare the entered password with the predefined password.
5. If the password matches, allow access to the system.
6. If the password does not match, display an error message and terminate the program.
7. Proceed to the main menu.

Algorithm 2: Add New Diary Entry

1. Display the main menu.
2. Select the “Add New Entry” option.
3. Prompt the user to enter the date.
4. Prompt the user to enter diary content.
5. Open the diary file in append mode.
6. Store the date and content in the file in a structured format.
7. Close the file.
8. Display a confirmation message.
9. Return to the main menu.

Algorithm 3: View All Diary Entries

1. Select the “View All Entries” option from the menu.
2. Open the diary file in read mode.
3. Read each entry line by line.
4. Display all stored diary entries on the screen.
5. Close the file.
6. Return to the main menu.

Algorithm 4: Edit Diary Entry

1. Select the “Edit Entry” option from the menu.
2. Prompt the user to enter the date of the entry to be edited.
3. Open the original diary file in read mode.
4. Create a temporary file in write mode. 5. Read each entry from the original file.
6. If the entered date matches an existing entry, replace it with updated content.
7. Write all other entries unchanged into the temporary file.

8. Close both files.
9. Delete the original diary file.
10. Rename the temporary file as the diary file.
11. Display a success or failure message.
12. Return to the main menu.

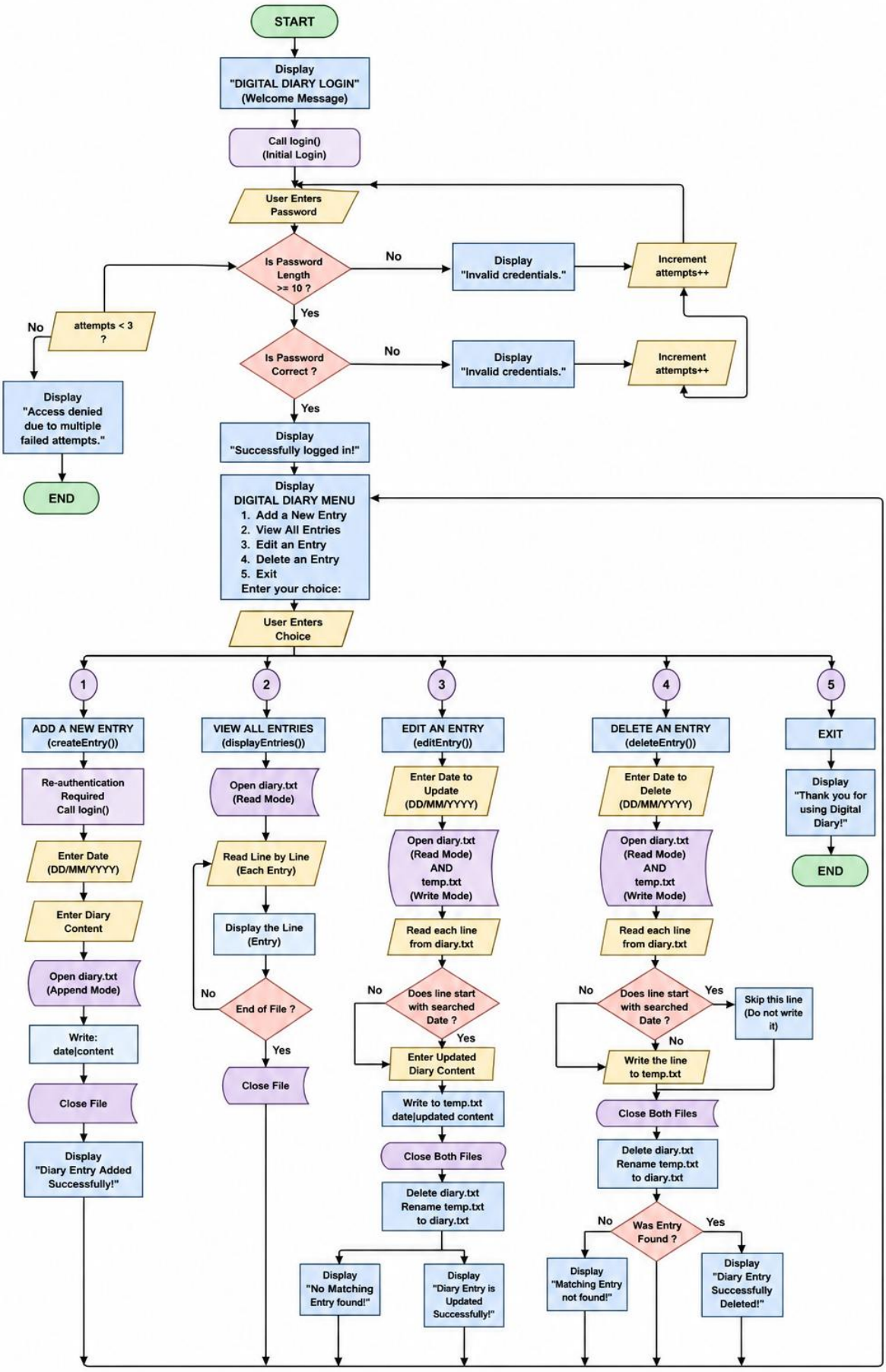
Algorithm 5: Delete Diary Entry

1. Select the “Delete Entry” option from the menu.
2. Prompt the user to enter the date of the entry to be deleted.
3. Open the original diary file in read mode.
4. Create a temporary file in write mode.
5. Read each entry from the diary file.
6. Skip the entry that matches the given date.
7. Write all remaining entries into the temporary file.
8. Close both files.
9. Remove the original diary file.
10. Rename the temporary file as the diary file.
11. Display deletion status message.
12. Return to the main menu.

Algorithm 6: Exit System

1. Select the “Exit” option from the menu.
2. Display a thank-you message.
3. Terminate the program.

FLOW CHART :


START
Display "DIGITAL DIARY LOGIN" (Welcome Message)
Call login() (Initial Login)
User Enters Password
Is Password Length >= 10 ?
No
Display "Invalid credentials."
Increment attempts++
Yes
Is Password Correct ?
No
Display "Invalid credentials."
Increment attempts++
Yes
attempts < 3 ?
No
Display "Access denied due to multiple failed attempts."
END
Display "Successfully logged in!"
Display DIGITAL DIARY MENU
1. Add a New Entry
2. View All Entries
3. Edit an Entry
4. Delete an Entry
5. Exit
Enter your choice:
User Enters Choice
1
ADD A NEW ENTRY (createEntry())
Re-authentication Required Call login()
Enter Date (DD/MM/YYYY)
Enter Diary Content
Open diary.txt (Append Mode)
Write: date|content
Close File
Display "Diary Entry Added Successfully!"
2
VIEW ALL ENTRIES (displayEntries())
Open diary.txt (Read Mode)
Read Line by Line (Each Entry)
Display the Line (Entry)
No
End of File ?
Yes
Close File
3
EDIT AN ENTRY (editEntry())
Enter Date to Update (DD/MM/YYYY)
Open diary.txt (Read Mode) AND temp.txt (Write Mode)
Read each line from diary.txt
No
Does line start with searched Date ?
Yes
Enter Updated Diary Content
Write to temp.txt date|updated content
Close Both Files
Delete diary.txt Rename temp.txt to diary.txt
Display "No Matching Entry found!"
Display "Diary Entry is Updated Successfully!"
4
DELETE AN ENTRY (deleteEntry())
Enter Date to Delete (DD/MM/YYYY)
Open diary.txt (Read Mode) AND temp.txt (Write Mode)
Read each line from diary.txt
No
Does line start with searched Date ?
Yes
Skip this line (Do not write it)
No
Write the line to temp.txt
Close Both Files
Delete diary.txt Rename temp.txt to diary.txt
No
Was Entry Found ?
Yes
Display "Matching Entry not found!"
Display "Diary Entry Successfully Deleted!"
5
EXIT
Display "Thank you for using Digital Diary!"
END

7.IMPLEMENTATION DETAILS

Program Structure

The Digital Diary Entry System is developed using a structured and modular programming approach. The program is divided into different functions, where each function performs a specific task. This approach improves the readability, maintainability, and clarity of the program.

The overall structure of the program includes:

• Header Files
• Class Definition
• Login Function
• Create Entry Function
• Display Entries Function
• Edit Entry Function
• Delete Entry Function
• Main Function with Menu-Driven Interface

The program execution begins from the main() function, where the user is first authenticated using the login function. After successful login, a menu is displayed repeatedly, allowing the user to perform diary operations until the Exit option is selected.

Language and Environment Used

Programming Language: C++

• Libraries Used:

<iostream> → Input and output operations
<fstream> → File handling (reading and writing diary entries)
<string> → Handling text data such as dates and diary content
<cstdio> → File operations like remove() and rename()

• Data Storage:

Diary entries are stored using a text file (diary.txt)

• Each Record Contains:

Date
Diary Content

Key Functional Modules

The Digital Diary System follows a simple and modular architecture to ensure clarity, maintainability, and ease of use. The system is divided into the following main modules:

A. Login Module:

- This module is responsible for user authentication.
- It prompts the user to enter a password before accessing the diary system.
- The entered password is verified with the stored password. ○ Access to the system is granted only if the password matches, ensuring data security.

B. Diary Management Module:

- This module handles all diary-related operations. ○ It allows users to add new diary entries with date and content. ○ Users can view all previously stored entries.
- It provides functionality to edit existing diary entries.
- It also allows deletion of unwanted or outdated entries. **C. File Management Module:**
- This module manages permanent storage of diary data.
- It handles file creation, reading, writing, and updating operations.
- Data is stored in a text file to ensure availability even after program termination.
- It ensures safe retrieval and modification of stored information.

Working of the Program:

- The program starts by displaying a login screen.
- The user is prompted to enter the password.
- If the entered password is correct, access to the main menu is provided.
- The main menu displays multiple options such as add entry, view entries, edit entry, delete entry, and exit.
- The user selects the required operation from the menu.
- Based on the user's choice, the corresponding function is executed.
- Any changes made by the user are updated in the file.
- The program continues to run until the user chooses to exit.

SOURCE CODE EXPLANATION

1.Header Files

```
#include <iostream>
#include <fstream>
#include <string>
#include <cstdio>
#include <cstdlib>
using namespace std;
```

<iostream>

- used for input and output operations
- Enables cin (input) and cout (output)

<fstream>

- Used for file handling
- Allows reading from and writing to files

<string>

- Allows usage of the string data type

<cstdio>

- Provides functions like remove() and rename() for file operations

<cstdlib>

- Used for exit() function to terminate the program

Using namespace std;

- Avoids writing std:: before standard functions and objects

2.Class Definition

```
class DiaryEntry
{
public:
   string date;
   string text;

   void createEntry();
   void displayEntries();
   void editEntry();
   void deleteEntry();
};
```

- Defines a class named DiaryEntry
- Represents one diary management system • public:
  - Makes class members accessible outside the class
- date stores the diary entry date • text stores the diary content • Function declarations for:
  - createEntry() → Adding entry o displayEntries() → Viewing entries o editEntry() → Editing entry o deleteEntry() → Deleting entry
- };
  - Ends the class definition

3.Login Function

```
void login()
{
   string pass;
   const string correctPassword = "digitalDiary";
   int attempts = 0;

   while (attempts < 3)
   {
      cout << "Enter Password: ";
      cin >> pass;

      if (pass.length() < 10)
      {
         cout << "Invalid credentials.\n";
         attempts++;
      }
```

```
        else if (pass != correctPassword)
        {
            cout << "Invalid credentials.\n";
            attempts++;
        }
        else
        {
            cout << "Successfully logged in!\n";
            return;
        }
    }

    cout << "Access denied due to multiple failed attempts.\n";
    exit(0);
}
```

- Function for password authentication
- Stores user-entered password
- Displays login prompt and reads password
- Checks whether the password is correct
- Stops the program if password is wrong

4.Create Diary Entry Function

```
void DiaryEntry::createEntry()
{
    cout << "\nRe-authentication required to add a new entry.\n";
    login();

    ofstream file("diary.txt", ios::app);

    cout << "\nEnter Date (DD/MM/YYYY): ";
    cin >> date;
    cin.ignore();

    cout << "Enter your Diary Content:\n";
    getline(cin, text);

    file << date << "|" << text << endl;
    file.close();

    cout << "Diary Entry Added Successfully!\n";
}
```

- Opens diary.txt in append mode

- New data is added without deleting old data
- Takes date input
- Clears input buffer before reading full text
- Reads complete diary content including spaces
- Stores date and content in file separated by |
- Each entry stored on a new line
- Closes the file safely
-

5.Display Diary Entries Function

```
void DiaryEntry::displayEntries()
{
   ifstream file("diary.txt");
   string line;

   cout << "\n------ STORED DIARY ENTRIES ------\n";
   while (getline(file, line))
   {
      cout << line << endl;
   }
   file.close();
}
```

- Opens file in read mode
- Reads the file line by line
- Displays each diary entry
- Closes the file

6.Edit Diary Entry Function

```
void DiaryEntry::editEntry()
{
   ifstream file("diary.txt");
   ofstream tempfile("temp.txt");
   string searchDate, line;
   bool found = false;

   cout << "Enter date to update (DD/MM/YYYY): ";
   cin >> searchDate;
   cin.ignore();

   while (getline(file, line))
   {
      if (line.find(searchDate) == 0)
```

```
        {
            found = true;
            cout << "Enter updated diary content:\n";
            getline(cin, text);
            tempfile << searchDate << "|" << text << endl;
        }
        else
        {
            tempfile << line << endl;
        }
    }

    file.close();
    tempfile.close();

    remove("diary.txt");
    rename("temp.txt", "diary.txt");

    if (found)
        cout << "Diary Entry is Updated Successfully!\n";
    else
        cout << "No Matching Entry found!\n";
}
```

- Original file opened for reading
- Temporary file created for updated data
- searchDate stores date to edit
- found checks whether entry exists
- Checks if the line starts with entered date
- Writes updated entry to temp file
- Deletes old file
- Renames temp file as main diary file

7.Delete Diary Entry Function

```
void DiaryEntry::deleteEntry()
{
    ifstream file("diary.txt");
    ofstream tempfile("temp.txt");
    string searchDate, line;
    bool found = false;

    cout << "Enter date to delete (DD/MM/YYYY): ";
    cin >> searchDate;

    while (getline(file, line))
```

```
    {
        if (line.find(searchDate) == 0)
        {
            found = true;
            continue;
        }
        tempfile << line << endl;
    }

    file.close();
    tempfile.close();

    remove("diary.txt");
    rename("temp.txt", "diary.txt");

    if (found)
        cout << "Diary Entry Successfully Deleted!\n";
    else
        cout << "Matching Entry not found!\n";
}
```

- if (line.find(searchDate)==0)
  {

```
found=true;
continue;
    }
```

- If date matches, entry is skipped (deleted)
- tempfile<<line<<endl;
- Writes remaining entries to temp file
- remove("diary.txt");
- rename("temp.txt", "diary.txt");
- Updates diary file after deletion

8.Main Function

```
int main()
{
    cout << "============================\n";
    cout << "   DIGITAL DIARY LOGIN\n";
    cout << "============================\n";

    login();   // initial login

    DiaryEntry manager;
    int option;

    while (true)
    {
        cout << "\n============================\n";
        cout << "   DIGITAL DIARY MENU\n";
        cout << "============================\n";
        cout << "1. Add a New Entry\n";
        cout << "2. View All Entries\n";
        cout << "3. Edit an Entry\n";
        cout << "4. Delete an Entry\n";
        cout << "5. Exit\n";
        cout << "Enter your choice: ";
        cin >> option;

        switch (option)
        {
        case 1:
            manager.createEntry();
            break;
        case 2:
            manager.displayEntries();
            break;
        case 3:
            manager.editEntry();
            break;
        case 4:
            manager.deleteEntry();
            break;
        case 5:
            cout << "Thank you for using Digital Diary!\n";
            exit(0);
        default:
            cout << "Invalid Choice! Please Try Again.\n";
        }
```

```
    }

    return 0;
}
```

- int main()
  Program execution starts here
- login();
  Calls login before accessing diary
- DiaryEntry manager;
  Creates object of DiaryEntry class
- while(true)
  Infinite loop to keep menu running
- switch(option) Handles menu choices
- Menu Operations
  Option Function
  1 Add Entry
  2 View Entries
  3 Edit Entry
  4 Delete Entry
  5 Exit
- exit(0);
  Ends the program safely
- return 0;
  Indicates successful execution

8.RESULTS AND DISCUSSIONS

OUTPUT SCREENS

```
PS D:\CPP FINALLL> g++ diary5.cpp -o diary5
PS D:\CPP FINALLL> ./diary5.exe
============================
   DIGITAL DIARY LOGIN
============================
Enter Password: digitalDiary
Successfully logged in!
```

```
==========================
   DIGITAL DIARY MENU
==========================
1. Add a New Entry
2. View All Entries
3. Edit an Entry
4. Delete an Entry
5. Exit
Enter your choice: 1

Enter Date (DD/MM/YYYY): 23022026
Enter your Diary Content:
OUR TEAM MEMBERS ARE HARSHA VARDHAN, POOJA, VARSHINI, BHAVAN
Diary Entry Added Successfully!
```

```
PS D:\CPP FINALLL> g++ diary5.cpp -o diary5
PS D:\CPP FINALLL> ./diary5.exe
============================
   DIGITAL DIARY LOGIN
============================
Enter Password: 0212
Invalid credentials.
Enter Password: 7896541230
Invalid credentials.
Enter Password: 0125478
Invalid credentials.
Access denied due to multiple failed attempts.
PS D:\CPP FINALLL>
```

```
==========================
   DIGITAL DIARY MENU
==========================
1. Add a New Entry
2. View All Entries
3. Edit an Entry
4. Delete an Entry
5. Exit
Enter your choice: 2

------STORED DIARY ENTRIES ------
21022026|WELCOME TO DIGITAL DIARY WORLD!!!
22022026|THIS IS BATCH 10 AND OUR C++ MINI PROJECT IS DIGITAL DIARY ENTRY SYSTEM...
23022026|OUR TEAM MEMBERS ARE HARSHA VARDHAN, POOJA, VARSHINI, BHAVAN
```

```
==========================
   DIGITAL DIARY MENU
==========================
1. Add a New Entry
2. View All Entries
3. Edit an Entry
4. Delete an Entry
5. Exit
Enter your choice: 1

Enter Date (DD/MM/YYYY): 20022026
Enter your Diary Content:
HELLO A VERY GOOD MORNING TO ALL!!!
Diary Entry Added Successfully!
```

```
==========================
   DIGITAL DIARY MENU
==========================
1. Add a New Entry
2. View All Entries
3. Edit an Entry
4. Delete an Entry
5. Exit
Enter your choice: 2

------STORED DIARY ENTRIES ------
21022026|WELCOME TO DIGITAL DIARY WORLD!!!
22022026|THIS IS BATCH 10 AND OUR C++ MINI PROJECT IS DIGITAL DIARY ENTRY SYSTEM...
23022026|OUR TEAM MEMBERS ARE HARSHA VARDHAN, POOJA, VARSHINI, BHAVAN
20022026|HELLO A VERY GOOD MORNING TO ALL!!!
```

```
==========================
   DIGITAL DIARY MENU
==========================
1. Add a New Entry
2. View All Entries
3. Edit an Entry
4. Delete an Entry
5. Exit
Enter your choice: 3
Enter date to update (DD/MM/YYYY): 20022026
Enter updated diary content:
HELLO! A VERY GOOD MORNING EVERYONE!!!
Diary Entry is Updated Successfully!

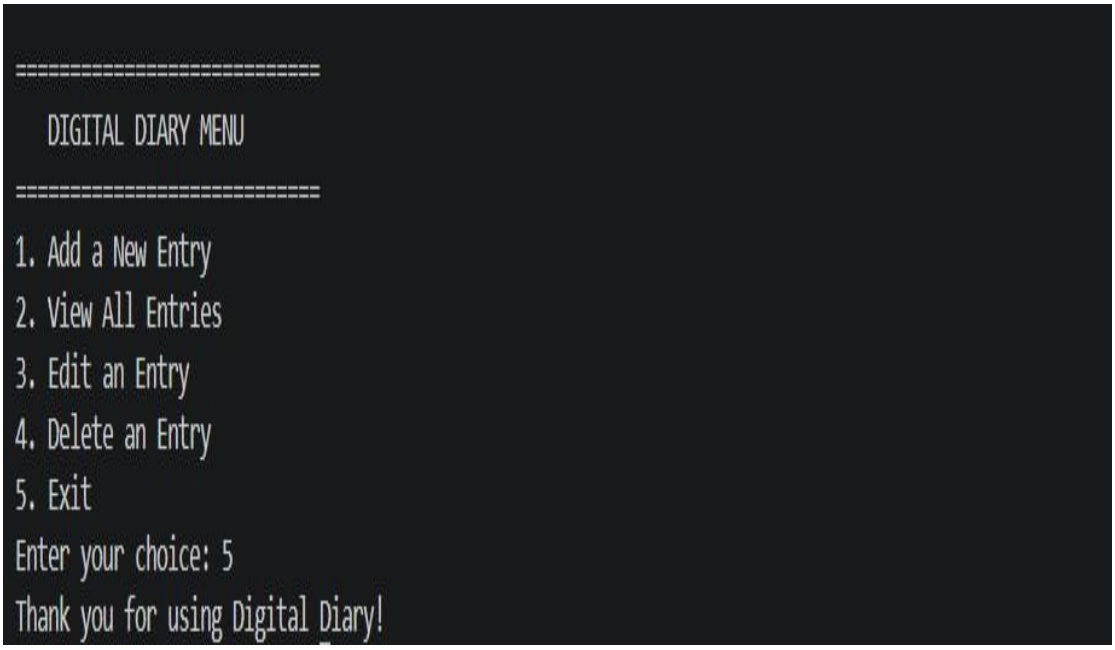
==========================
   DIGITAL DIARY MENU
==========================
1. Add a New Entry
2. View All Entries
3. Edit an Entry
4. Delete an Entry
5. Exit
Enter your choice: 2

------STORED DIARY ENTRIES ------
21022026|WELCOME TO DIGITAL DIARY WORLD!!!
22022026|THIS IS BATCH 10 AND OUR C++ MINI PROJECT IS DIGITAL DIARY ENTRY SYSTEM...
23022026|OUR TEAM MEMBERS ARE HARSHA VARDHAN, POOJA, VARSHINI, BHAVAN
20022026|HELLO! A VERY GOOD MORNING EVERYONE!!!
```

```
==========================
   DIGITAL DIARY MENU
==========================
1. Add a New Entry
2. View All Entries
3. Edit an Entry
4. Delete an Entry
5. Exit
Enter your choice: 4
Enter date to delete (DD/MM/YYYY): 20022026
Diary Entry Successfully Deleted!

==========================
   DIGITAL DIARY MENU
==========================
1. Add a New Entry
2. View All Entries
3. Edit an Entry
4. Delete an Entry
5. Exit
Enter your choice: 2

------STORED DIARY ENTRIES ------
21022026|WELCOME TO DIGITAL DIARY WORLD!!!
22022026|THIS IS BATCH 10 AND OUR C++ MINI PROJECT IS DIGITAL DIARY ENTRY SYSTEM...
23022026|OUR TEAM MEMBERS ARE HARSHA VARDHAN, POOJA, VARSHINI, BHAVAN
```





9.CONCLUSION

The Digital Diary Entry System was successfully developed using C++ to provide a simple and secure way of maintaining personal diary records. The system allows users to add, view, edit, and delete diary entries efficiently through a menu-driven interface. File handling ensures permanent storage of data, reducing the risk of data loss.
The project demonstrates the practical application of object-oriented programming and file handling concepts. Although the system is console-based and designed for a single user, it meets its objectives effectively and can be enhanced further with advanced features in the future.

LIMITATIONS

- Supports only single-user access.

- Uses basic password authentication.
- Console-based interface with limited interaction.
- Stores data in text files, not a database.
- No advanced search or backup features.

FUTURE EXTENSIONS

- Add multi-user login support.
- Implement strong security and encryption.
- Use database storage instead of text files.
- Develop a graphical user interface (GUI).
- Include search, backup, and cloud storage features.

10.REFERENCES

- GeeksforGeeks.
  "File Handling and Object-Oriented Programming in C++." https://www.geeksforgeeks.org
- W3Schools.
  "C++ Basics and File Handling." https://www.w3schools.com
- Object-Oriented Programming with C++. E. Balagurusamy, *Object-Oriented Programming with C++*, McGraw Hill Education.

11.APPENDIX

SOURCE CODE:

```
#include <iostream>
#include <fstream>
#include <string>
#include <cstdio>
#include <cstdlib>
using namespace std;

class DiaryEntry
{
public:
string date;
string text;

void createEntry();
void displayEntries();
void editEntry();
void deleteEntry();
```

```
};

void login()
{
string pass;
const string correctPassword = "digitalDiary";
int attempts = 0;

while (attempts < 3)
{
cout << "Enter Password: ";
cin >> pass;

if (pass.length() < 10)
{
cout << "Invalid credentials.\n";
attempts++;
}

else if (pass != correctPassword)
{
cout << "Invalid credentials.\n";
attempts++;
}
else
{
cout << "Successfully logged in!\n";
return;
}
}
cout << "Access denied due to multiple failed attempts.\n";
exit(0);
}

void DiaryEntry::createEntry()
{
cout << "\nRe-authentication required to add a new entry.\n";
login();

ofstream file("diary.txt", ios::app);

cout << "\nEnter Date (DD/MM/YYYY): ";
cin >> date;
cin.ignore();
cout << "Enter your Diary Content:\n";
getline(cin, text);
```

```
file << date << "|" << text << endl;
file.close();
cout << "Diary Entry Added Successfully!\n";
}

void DiaryEntry::displayEntries()
{
ifstream file("diary.txt");
string line;

cout << "\n------ STORED DIARY ENTRIES ------\n";
while (getline(file, line))
{
cout << line << endl;
}
file.close();
}

void DiaryEntry::editEntry()
{
ifstream file("diary.txt");
ofstream tempfile("temp.txt");
string searchDate, line;
bool found = false;

cout << "Enter date to update (DD/MM/YYYY): ";
cin >> searchDate;
cin.ignore();

while (getline(file, line))
{
if (line.find(searchDate) == 0)
{
found = true;
cout << "Enter updated diary content:\n";
getline(cin, text);
tempfile << searchDate << "|" << text << endl;
}
else
{
tempfile << line << endl;
}
}

file.close();
```

```
tempfile.close();

remove("diary.txt");
rename("temp.txt", "diary.txt");

if (found)
cout << "Diary Entry is Updated Successfully!\n";
else
cout << "No Matching Entry found!\n";
}

void DiaryEntry::deleteEntry()
{
ifstream file("diary.txt");
ofstream tempfile("temp.txt");
string searchDate, line;
bool found = false;

cout << "Enter date to delete (DD/MM/YYYY): ";
cin >> searchDate;

while (getline(file, line))
{
if (line.find(searchDate) == 0)
{
found = true;
continue;
}
tempfile << line << endl;
}

file.close();
tempfile.close();

remove("diary.txt");
rename("temp.txt", "diary.txt");

if (found)
cout << "Diary Entry Successfully Deleted!\n";
else
cout << "Matching Entry not found!\n";
}

int main()
{
cout << "============================\n";
```

```
cout << "   DIGITAL DIARY LOGIN\n";
cout << "============================\n";

login();   // initial login

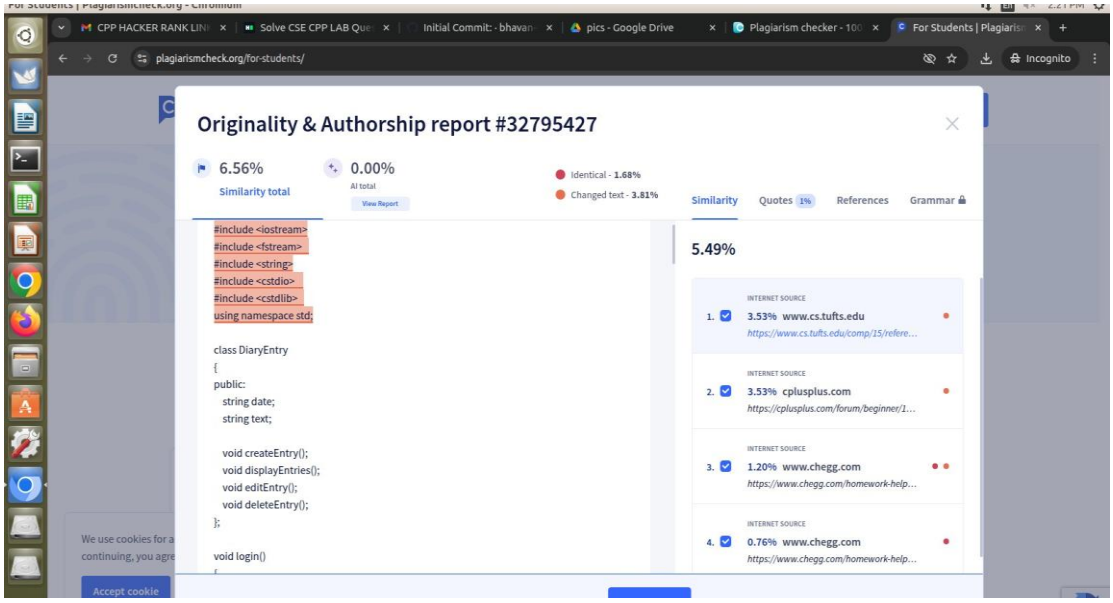
DiaryEntry manager;
int option;

while (true)
{
cout << "\n============================\n";
cout << "   DIGITAL DIARY MENU\n";
cout << "============================\n";
cout << "1. Add a New Entry\n";
cout << "2. View All Entries\n";
cout << "3. Edit an Entry\n";
cout << "4. Delete an Entry\n";
cout << "5. Exit\n";
cout << "Enter your choice: ";
cin >> option;

switch (option)
{
case 1:
manager.createEntry();
break;
case 2:
manager.displayEntries();
break;
case 3:
manager.editEntry();
break;
case 4:
manager.deleteEntry();
break;
case 5:
cout << "Thank you for using Digital Diary!\n";
exit(0);
default:
cout << "Invalid Choice! Please Try Again.\n";
}
}

return 0;
}
```

PLAG REPORT


Originality & Authorship report #32795427
6.56%
Similarity total
0.00%
AI total
Identical - 1.68%
Changed text - 3.81%
Similarity
Quotes 1%
References
Grammar
5.49%
#include <iostream>
#include <fstream>
#include <string>
#include <cstdio>
#include <cstdlib>
using namespace std;
class DiaryEntry
{
public:
string date;
string text;
void createEntry();
void displayEntries();
void editEntry();
void deleteEntry();
};
void login()
1. 3.53% www.cs.tufts.edu
https://www.cs.tufts.edu/comp/15/refere...
2. 3.53% cplusplus.com
https://cplusplus.com/forum/beginner/1...
3. 1.20% www.chegg.com
https://www.chegg.com/homework-help...
4. 0.76% www.chegg.com
https://www.chegg.com/homework-help...